

Инструкция по работе с картами и типовыми формами

Общие сведения (карты)

Карта сбора данных представляет собой JSON документ, в котором описан набор полей, доступных для реестра, для которого предназначена карта, определен механизм компоновки данной карты с другими, а также набор общих полей.

Структура карты

Карты сбора данных описываются в документах формата JSON определенной структуры. Структура документа описана с помощью JSON schema, с ней можно ознакомиться в файлах `mapping.endpoint.schema`, `mapping.extension.schema`, `mapping.property.schema` в корневом каталоге git-хранилища.

Пример описания карты:

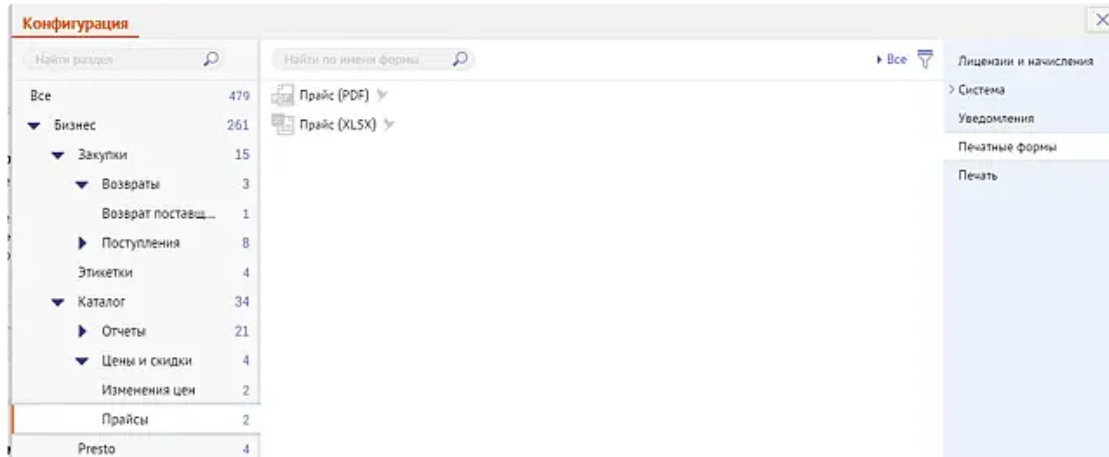
```
1  {
2      "__format__": "0.1-DRAFT",
3      "Name": "Командировка",
4      "Title": "Сотрудники / Графики работ / Командировки",
5      "InternalName": "BusinessTripMapping",
6      "Type": 0,
7      "Bases": [
8          "BaseMapping"
9      ],
10     "Extensions": [
11         "OurOrganizationMapping",
12         "HeadOrganizationMapping",
13         "ConsentListMapping"
14     ],
15     "ComponentID": ["c00cfd6f-8b5a-4ce1-80da-cdd7195d6923", ...],
16     "Data": {
17         "3": [1, "Идентификатор", ... , null, null, []],
18         "4": [5, "Документ", ... , null, null, [ 1]],
19         ...
20         "55": [169, "Сотрудник.Руководитель.Отчество", ... , null, null,
21     []],
22         "56": [170, "Сотрудник.Руководитель.Должность", ... , null, null,
23     []]
24     }
25 }
```

- **__format__** – служебное, необязательное свойство. Определяет версию формата карты (набор полей, их структуру и обработку). Если не указано, будет взят текущий формат по умолчанию. По мере улучшения удобства и оптимизации структуры карт будут появляться новые версии форматов. Поле служит для того, чтобы не переходить во всех картах на новый формат одновременно, старые форматы продолжают поддерживаться. Если структура карты не совпадет с указанным форматом (или форматом по умолчанию, если не указано), то карта не будет импортирована в сервис печатных форм.
- **Name** – «эндпоинт» или служебное имя реестра, для которого создана карта. Для реестров ЭДО совпадает с типом документа. Например, «СчетИсх», «ОстаткиПоПартиям».
- **Title** – Понятное, удобное имя реестра или раздела, в котором хранятся соответствующие формы, созданные по этой карте. Вложенность разделов обозначается с помощью разделителя «/». Формы, созданные по этой карте, будут располагаться в общем списке печатных форм («Конфигурация» / «Печатные формы») с использованием этого свойства.

Например:

"Бизнес / Закупки / Возвраты"

"Бизнес / Каталог / Цены и скидки / Прайсы"



- **Lang** – локализованное имя реестра/раздела. Представляет из себя словарь с ключом-языком локализации и значением в виде локализованного **Title** для соответствующего языка. Например: `{"en": "Business / Sales and CRM / CRM / Deals"}`
- **InternalName** – служебное имя карты, в общем случае совпадает с именем файла. Сейчас необходимо, но в будущем может быть удалено.
- **Type** – тип карты, эндпоинт или расширение (описание компоновки карт см. ниже).
 - **0** – Конечные карты («эндпоинты»).
 - **1** – Карты расширения.
- **Bases** – список базовых карт (описание компоновки карт см. ниже).
- **Extensions** – список карт расширений (описание компоновки карт см. ниже).
- **Zone** – идентификатор участка прав, при наличии полного доступа на который, можно добавлять, изменять и удалять формы. Список участков можно уточнить, например, в карточке сотрудника с неполными правами (поле **Identifier** метода чтения структуры участков прав)
- **ComponentID** – список UUID идентификаторов компонент (**обязателен**). Его нужно узнавать у ответственного за функционал, чтобы избежать разночтений. Идентификатор можно найти в хранилище компонентов в [сма](https://git.sbis.ru/app-market/applications) (<https://git.sbis.ru/app-market/applications>) или в к консоли при открытии компонента в ответе от метода `Read \ ReadByComponentUUID` (Настройки-Компоненты)
- **Data** – набор полей карты для вставки в печатную форму (описание полей карт см. ниже).

Компоновка карт

Если вы проектируете новый сервис, который предполагает функционал печатных форм или шаблонов писем, то следование этим рекомендациям позволит вам сэкономить время при подключении пользовательских печатных форм.

1. Постарайтесь, чтобы ваши методы возвращали данные в простом формате (в виде `Record`, `RecordSet` или простые типы данных). Использование `JSON`, `XML` или строк вашего собственного формата, равно как и глубоко вложенные `RecordSet`-ы усложнят вам процедуру составления карты или даже потребуют разработку новых методов, удобных для использования в картах.
2. Старайтесь не делать методы, которые возвращают сразу много разнородной информации (тем более, если они собирают данные из нескольких других и разными вызовами). Это создаст лишнюю нагрузку на ваш сервис, в том случае, если в печатных формах используется только часть возвращаемых данных. Конечно, если все данные у вас лежат в одной таблице или их возврат в одном методе для вас условно «бесплатен», то можете не следовать этой рекомендации.
3. Если у вашего сервиса несколько карт, то нет смысла копировать часть одинаковых полей из карты в карту. Можно оформить отдельную карту для группы связанных полей и включать ее как подкарту во все остальные.

Из приятных бонусов следует учитывать, что для разных версий основного сервиса могут отдаваться разные карты, т.к. репозиторий с картами живет по вехам онлайн. Это позволяет менять методы, описанные в картах, не опасаясь, что сломается совместимость.

В конечном итоге, грамотно спроектированная карта, должна выглядеть примерно так (для пользователя):

Найти...

Общая информация

Текущий пользователь

Дата печати

Документ

Номенклатура

Путь к регламенту

Автор

Подписант

Договор

Ответственный

Аккаунт

Расчетный счет

Связанные документы

Документы-основания

Документы-следствия

Скидки

Есть в биллинге

Номер

Дата создания

Ссылка

Ссылка доступна

Сумма

Сумма без НДС

Оплачено

Название регламента

Комментарий

Найти...

Комментарий

Скидка

Официальные лица

Руководитель

Главный бухгалтер

Наша организация

Название

Основной расчетный счет

Адрес

Контакты

Директор

Главный бухгалтер

Кассир

Регистрация

Представители

Дополнительно

Примечание

Код

Логотип

Дата регистрации

Является ИП

Печать

Использовать НДС

Головная организация

Название

Основной расчетный счет

Скопировать

Основной расчетный счет

Адрес

Контакт

Директор

Главный бухгалтер

Кассир

Регистрация

Представители

Дополнительно

Примечание

Логотип

Дата регистрации

Печать

Контрагент

Название

ФИО

Удостоверение личности

Основной расчетный счет

Адрес

Контакт

Директор

Директор ЕРЮЛ

Регистрация

Дополнительно

Представители

Статус ЭДО

Дополнительно

Представители

Статус ЭДО

Примечание

Пол

Дата рождения

Адрес места рождения

Внешний идентификатор

Логотип

Является ИП

Лента событий

Вложения

Исполнитель

Должность

Переход

Этап

Комментарий

Список вложений

Дата

Связанные документы

Идентификатор документа

Тип документа

Идентификатор регламента

Название регламента

Номер

Дата создания

Найти...

Комментарий

Скопировать

Скидка

Официальные лица

Руководитель

Главный бухгалтер

Наша организация

Название

Основной расчетный счет

Адрес

Контакты

Директор

Главный бухгалтер

Кассир

Регистрация

Представители

Дополнительно

Примечание

Код

Логотип

Дата регистрации

Является ИП

Печать

Использовать НДС

Головная организация

Название

Основной расчетный счет

Найти...

Основной расчетный счет

Адрес

Контакт

Директор

Главный бухгалтер

Кассир

Регистрация

Представители

Дополнительно

Примечание

Логотип

Дата регистрации

Печать

Контрагент

Название

ФИО

Удостоверение личности

Основной расчетный счет

Адрес

Контакт

Директор

Директор ЕГРЮЛ

Регистрация

Дополнительно

Представители

Статус ЭДО

Найти...

Дополнительно

Представители

Статус ЭДО

Примечание

Пол

Дата рождения

Адрес места рождения

Внешний идентификатор

Логотип

Является ИП

Лента событий

Вложения

Исполнитель

Должность

Переход

Этап

Комментарий

Список вложений

Дата

Связанные документы

Идентификатор документа

Тип документа

Идентификатор регламента

Название регламента

Номер

Дата создания

Все эти поля и таблицы можно использовать в документе.

Карта должна проектироваться таким образом, чтобы отображать реальное описание объектов БЛ, а не такое, которое завязано на ваши методы. Например, если у вас есть один метод, который возвращает сразу информацию и по конкретному поступлению товаров и по поставщику этих товаров, то в карте необходимо разделить группу полей не две секции ("Документ.Поступление" и "Документ.Контрагент"):

- Документ.Поступление.Дата
- Документ.Поступление.Номенклатура
- Документ.Контрагент.Наименование
- Документ.Контрагент.ИНН

При проектировании методов старайтесь разделить их на несколько, логически наиболее несвязанных по данным. Это не будет ложиться дополнительной нагрузкой на ваш сервис при печати большого количества отчетов.

Разработать, для ваших потребностей, карту сбора данных можем и мы. В этом случае вам необходимо создать задачу на [Соборникова Евгения](#), где подробно расписать необходимый набор полей, который должен быть в ваших картах и какие методы возвращают эти поля.

В большинстве случаев мы готовы разработать карту в режиме «в план работ следующего месяца», т.е. не заставим вас долго ждать.

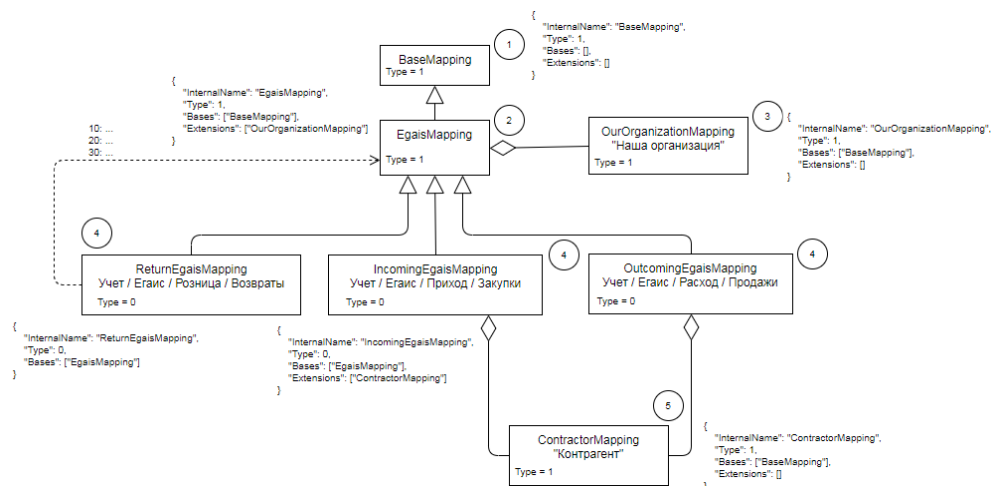
Следует учитывать, что тестирование карт это зона ответственности тестировщика своей предметной области. Мы можем дать вам набор вспомогательных инструментов для облегчения работы, например, скрипт генерации шаблона включающего все поля указанной карты.

Некоторые карты могут содержать группу общих полей, объединенных по смыслу, для включения в другие карты. Они не привязаны к какому-либо реестру, и используются как контейнеры полей. Примерами таких карт могут быть карты «Наша организация» (OurOrganizationMapping) и «Контрагент» (ContractorMapping). Некоторые могут содержать базовый набор полей и служить основой для других, уточняющих карт. Таким образом, несколько карт могут быть скомпонованы с помощью иерархии или агрегации, образуя общую конечную карту.

Основное отличие двух видов компоновки заключается в порядке добавления полей в конечную карту. Поля добавляются **рекурсивно**. Сначала обрабатывается базовая карта, затем добавляются (или вставляются в соответствующие индексы) поля текущей карты, после этого в конец добавляются поля из карт-расширений. Каждая из карт из коллекций **Bases** и **Extensions** обрабатывается по тому же самому алгоритму. Если одна из вложенных карт уже была обработана, она пропускается, и повторно не обрабатывается.

Это позволяет избежать дублирования одних и тех же полей в разных картах, а также изменять общие поля сразу для многих конечных карт. В общем случае все карты наследуются от базовой карты **BaseMapping**, в которой содержится общая информация для использования в печатных формах. Исходя из вышеуказанного существует два типа карт, что и определяется свойством **Type**.

Пример существующей компоновки:



Набор полей карты

В свойстве **Data** карты хранится набор полей для сбора данных. Это свойство может быть двух типов: **JSON array** или **JSON object**. В первом случае поля карты будут добавляться по порядку, как есть, в конец текущего уровня иерархии (добавление происходит рекурсивно, см. раздел «Компоновка карт»). Во втором случае в качестве ключа для поля можно указать индекс, по которому поле будет вставлено в текущий уровень иерархии. Порядок полей в карте имеет значение (см «Дополнительные сведения», п.2).

Структура поля

Каждое поле в карте состоит из элементов, количество и тип которых должны строго соответствовать структуре указанного формата. Для текущего формата «0.1-DRAFT» их 14.

Список и порядок элементов в карте:

1. **Id** (integer) – идентификатор поля, используемый для сбора данных. Должен быть уникальным в пределах карты, включая вложенные карты.
2. **Title** (string) – имя, с которым поле будет отображаться в списке полей и подставляться в форм. Должно быть уникальным в пределах карты.
3. **Name** (string) – имя поля в Record, из которого берутся для данного поля, или имя, с которым поле помещается в Record для вызова какого-либо метода.
4. **View** (string) – тип отображения поля. Может быть:
 - value – скалярное значение.
 - list – таблица.
5. **Type** (string) – тип данных поля. Может быть:
 - Int – целое число.
 - Float – вещественное число.
 - Record – запись.
 - AsIs – «как есть», может использоваться только при сборе данных.
 - Text – строка.
 - Enum – перечисление.
 - Bool – логическое.

- Array – массив.
- Navigation – навигация, используется для вызова списочных методов.
- Sorting – сортировка, используется для вызова списочных методов.
- Money – деньги.
- Date – дата.
- Time – время.
- DateTime – дата/время.
- RecordSet – таблица, этот тип используется для вложенных таблиц.
- Signature – подпись, представляющая собой картинку.
- Image – картинка.
- Print – печать, представляющая собой картинку.
- Link – ссылка.
- TimeInterval – интервал времени.
- Identifier – поле-идентификатор. Используется в полях иерархии.

6. **Parent** (integer) – идентификатор поля-родителя для текущего.

7. **Endpoint** (string|null) – имя вызываемого метода для получения данных.

Имеет вид: **[Имя сервиса.]Имя объекта.Имя метода.**

Если имя сервиса не указано, то по умолчанию считается основной.

8. **Visible** (boolean) – флаг видимости поля в списке поле в редакторе печатных форм.

9. **Value** (null|string|number|boolean) – значение по умолчанию, которое подставляется, если значение поля null или не было получено. Значение по умолчанию берется только если, сбор данных не дошел до свойства: упал метод, который возвращает поле, или он не был вызван, потому что входящие параметры не инициализированы. Если метод вернул значение поля, то берется возвращенное значение (в том числе пустое).

10. **Parent_** (boolean) – флаг, показывающий что для поля есть дочерние.

11. **Preview** (string|null) – значение поля, выводимое в предварительном просмотре формы. Для таблиц задается массивом массивов в поле самой таблицы или массивами в полях-колонках.

12. **Substitution** (string|null) – имя подстановки формализованного документа. Не используется.

13. **Options** (null) – дополнительные опции обработки поля. Используется для специфических обработок полей, например, для того чтобы сумма возвращалась с 3 знаками после запятой ({"Appearance": {"Precision": 3}})

14. **Parameters** (array(integer)) – параметры, используемые при вызове метода. Задаются массивом с идентификаторами полей карты, которые будут использованы при вызове.

Дополнительные сведения

При работе с картами нужно учитывать следующие нюансы:

1. Последовательность значений для строки в превью таблицы должно совпадать с последовательностью полей в таблице по количеству и типу.
2. Поля, которые используются для вызова метода, не могут идти позже полей, которые из этого метода могут быть получены (порядок полей важен!).
3. Превью для таблицы представляет собой массив массивов.
4. Тип для таблиц первого уровня указывать не надо.
5. Превью перечисления представляет собой его формат.
6. Endpoint карты должен быть уникальным, как и ее Title.
7. Для задания иерархии для таблиц в ее поля нужно добавить:
 1. __HIERARCHY_SECTION_ID – идентификатор записи.
 2. __HIERARCHY_PARENT_ID – идентификатор родителя.
 3. __HIERARCHY_SECTION_FLAG – флаг, что является родителем.
8. Поле с типом AsIs не может быть видимым.
9. Если сделать невидимым родительское поле, то станут невидимыми и все дочерние

в списке полей в редакторе.

10. Нельзя создавать циклы при вызове полей – они будут отсекаются.
11. В именах полей запрещено использовать символы: <, >, / .
12. Превью для вложенных таблиц представляют собой 3-х уровневый массив.
13. Превью полей таблицы должны быть заполнены корректно.
14. Вложенные таблицы 2 уровня (таблицы 3 уровня) нельзя использовать.
15. Title для поля рекомендуется задавать в соответствии с иерархией полей. Исключения составляют скрытые поля.
16. Title должен быть задан так, чтобы его значение было понятно любому пользователю. Сокращения разрешены только общепринятые. Например, корр счет, ИНН, КПП. Исключения составляют скрытые поля.
17. Для того, чтобы в поле карты попал идентификатор переданного документа, в Params нужно указать \$ID.
18. Если при отсутствии параметров поле возвращает ошибку, необходимо в Value полей-аргументов добавить значения по умолчанию, передача которых не вызовет ошибку в вызываемом методе.
19. Необходимо внимательно задавать тип поля, во избежание ошибок или предупреждений в логах.
20. Id для поля можно задавать в пределах от 1 до 99999.
21. Чтобы получить значение из поля json/словарь/массив, в «Name» после названия поля нужно указать селектор. Он вводится в квадратных скобках, при этом для обозначения вложенности используется символ «/», а для получения элемента массива — указывается его индекс.

Например: метод возвращает поле "JsonField" с содержимым: {"Folder1": {"Name": "123"}, "SomeArray": ["Foo", "Bar"]}. Тогда примеры селекторов могут быть такие: "JsonField[Folder1/Value]" или "JsonField[SomeArray/1]".

Аналогично можно обратиться к определенному Record'у внутри Recordset.

Например: метод возвращает Recordset "Подписанты": [{"Фамилия": "Иванов"}, {"Фамилия": "Петров"}]. Чтобы получить первого подписанта, в Name необходимо указать: "Подписанты[0/Фамилия]".

Репозиторий хранения карт

Инсталляционные формы и карты свойств хранятся в git-репозитории: <https://git.sbis.ru/printforms/templateeditor-data>

Для изменения необходимых карт или шаблонов, необходимо мощью git клонировать себе репозиторий и отредактировать соответствующие файлы. Обновление данных в репозитории происходит через оформление merge request.

Проверка корректности карт

В корневом каталоге репозитория имеется python-скрипт validate.py для проверки корректности составленных карт. Валидация карт основана на JSON schema. Для запуска скрипта необходимо иметь на машине интерпретатор python, а так же установить python-модуль «jsonschema».

```
>pip
```

```
install
```

```
jsonschema
```

Для запуска проверки необходимо выполнить команду:

```
>python
```

```
validate.py
```

На данный момент реализована проверка карты на типы данных и количество элементов полей. Скрипт валидации регулярно расширяется. На текущий момент в репозиторий внедряем и pre-push hook, который выводит корректные сообщения, если были доброшены какие-то неправильные карты.

Создание карты

Для создания карты необходимо:

1. В папке **mappings** создать файл расширения **map**, с соответствующим названием по имени реестра.
2. Описать общую структуру карты (по информации выше). В нее должны входить все необходимые поля (Name, Title и т.д.)

Если для вызова метода используется идентификатор документа, то нужно описать поле с ним,

в **Value** которого должно стоять значение **\$ID**. Если используемых параметров больше, то их нужно описать как поля без заполненного поля **Name** и передавать их через **ExactValues**

(при вызове методов печати) с теми же названиями как в поле **Title**.

```
"Data": [
  [1, "Идентификатор", "IdO", "value", "Int", null, "$ID", false, "", false, null, null, null, []],
  [2, "ИмяМетодаподразделение", "ИмяМетода", "value", "Text", null, null, false, "", false, null, null, null, []],
  [6, "ИдОНашаОрганизация", "ПодразделениеНашаОрганизация", "value", "Text", 5, null, false, "0", false, null, null, null, []]
```

Для создания раздела с вложенными полями необходимо описать поле с **Parent@ = true**. После можно описывать дочерние свойства. Для них поле **Parent** должно быть равно идентификатору поля-родителя, а также в поле **Title** прописать Title родителя точка Title дочернего поля.

```
[211, "Документ.Сотрудник", null, null, null, 5, null, true, "", true, null, null, null, []],
[212, "Документ.Сотрудник.фамилия", "Сотрудник.ЧастноеЛичо.фамилия", "value", "Text", 211, null, true, "", false, "Иванов", null, null, []],
[213, "Документ.Сотрудник.Имя", "Сотрудник.ЧастноеЛичо.Имя", "value", "Text", 211, null, true, "", false, "Иван", null, null, []],
[214, "Документ.Сотрудник.Отчество", "Сотрудник.ЧастноеЛичо.Отчество", "value", "Text", 211, null, true, "", false, "Иванович", null, null, []],
[215, "Документ.Сотрудник.Подразделение", "Подразделение.Название", "value", "Text", 211, null, true, "", false, "Отдел 1", null, null, []],
```

Создание таблицы осуществляется аналогичным способом как и создание раздела с вложенными полями. Отличия в том что родительское поле будет иметь **View = list** и в его поле **Preview** содержится все содержимое таблицы в виде массивов.

```
[354, "Документ.Вехи", "Вехи", "list", null, 5, null, true, "", true, "[[2, \"\Bexa1\", \"2016-01-01\", [1, \"\Bexa2\", \"2016-03-01\"],",
[355, "Документ.Вехи.Состояние", "Состояние", "value", "Int", 354, null, true, "", false, "0", null, null, []],
[356, "Документ.Вехи.Название", "ДокументРасширение.Название", "value", "Text", 354, null, true, "", false, "Беха", null, null, []],
[357, "Документ.Вехи.Дата", "Дата", "value", "Date", 354, null, true, "", false, "2016-01-01", null, null, []],
```

Для вызова метода необходимо его имя написать в поле **Endpoint**, а идентификаторы полей параметров перечислить в поле **Params**. В **Params** могут быть переданы только идентификаторы полей. Чтобы получить из метода одно поле его имя прописывается в поле **Name**.

Если метод возвращает:

1. Скаляр – **Name** указывать не нужно.
2. Record – необходимо поле с методом сделать разделом.
3. RecordSet – нужно описать поле по аналогии с табличным.

```
[10, "Совещание", null, null, null, null, "Meeting.PrintData", true, "", true, null, null, null, [ 1]],
[11, "Совещание.Название", "Документ.Name", "value", "Text", 10, null, true, "", false, "Совещание", null, null, []],
[111, "Совещание.Дата создания", "Документ.CreateDate", "value", "DateTime", 10, null, true, "", false, "", null, null, []],
[2501, "Совещание.Ссылка", null, "value", "Link", 10, "CustomDataCollector.GetDocUrl", true, "", false, "", null, null, []],
[13, "Совещание.Дата начала", "Документ.DateTimeStart", "value", "DateTime", 10, null, true, "", false, "", null, null, []],
```

Если у поля указан Name, то оно будет считываться из ближайшего вышестоящего по дереву вызова метода. Константы описываются как поля с **Value** равным необходимому значению. Примеры карт можно найти здесь: <https://git.sbis.ru/printforms/templateeditor-data/tree/development/mappings>

Оформление merge request

При оформлении merge request необходимо указать хранилище **printforms/templateeditor-data** и указать нужную ветку для мержа. Именованние веток производится в соответствии с техпроцессом работы с git, утвержденным в компании.

Сведения о клиенте

Анализ ошибки (разра

Что обновлять

Автоматическое создание доброски

Хранилище: printforms/templateeditor-data

Ветка: 3.18.150/bugfix/sorted_pageparams

Заголовок: Ошибка №1175103715 от 2018-04-03 Кульков М.С.

Описание: http://online.sbis.ru/opendoc.html?guid=8c4e8805-3804-442e-a378-69618b7bd880 Необходимо отключить авто-импорт при старте сервиса. Полностью перейти на импорт из пакета

Create merge

В списке «Что обновлять» необходимо выбирать пункт **printing-templates-data-package**. Все остальные поля заполняются аналогично обычному MR.

+ svn + git + git branch Проверить

Merge request	https://git.sbis.ru/printforms/templateeditor-data/merge_requests/1408		
Merge request	https://git.sbis.ru/printforms/templateeditor-data/merge_requests/1409		
Merge request	https://git.sbis.ru/printforms/templateeditor-data/merge_requests/1407		

Что обновлять: printing-templates-dat... ☐ Требуется конвертация базы данных
printing-templates-data-package

Что протестировать:

Далее МР уходит на review ответственному за хранилище, и затем на сборку. Сразу после сборки, по событию из админки, происходит разворот пакета данных в сервис пользовательских печатных форм.

Инструкция по работе с типовыми формами

Общие сведения

Типовые формы представляют из себя 2 файла: json с параметрами формы и файл содержимого. Для pdf это html файл, для word – docx, для excel – xlsx. Эти файлы расположены в папке **templates** в том же репозитории где хранятся карты. Структура папки представляется из себя список каталогов с названиями карт, в каждом из которых расположены разделы с названиями форм, содержащие в себе файлы, информация о которых указана выше.

Типовые формы отличаются от пользовательских и типовых измененных флагом Basic = True.

Доброска форм производится по аналогии с картами.

Структура файла параметров типовых форм

__format__ - аналогично с картами.

TemplateID - уникальный идентификатор формы. Для новой формы задается случайным образом.

Name - название формы.

ExternalID - уникальный строковый идентификатор формы.

PrintDialog - имя контрола, который вызывается перед генерацией формы.

Mode - режим структуры документа при печати. **0** - документ, **1** - список, **2** - ценник.

Parent - идентификатор типовой формы, от которой была создана текущая. Для типовой формы имеет всегда значение **null**.

Basic - флаг, обозначающий что форма является типовой.

Editable - флаг возможности редактирования формы.

Shared - флаг, что форма является "расшаренной". **Не используется**, значение **false**.

Embedded - флаг, хранящий в себе сведения что форма является оберточной. Оберточная форма - форма, внутри которой может быть расположены другие формы.

Clouds - список облаков, в которых форма будет отображаться. **RU** - Россия, **KZ** - Казахстан, **UZ** - Узбекистан

PageParams - набор параметров, содержащий в себе ряд параметров для отображения документа. Применяются только для pdf. В нем содержатся параметры:

- **FontFamily** - шрифт по умолчанию.
- **FontSize** - размер шрифта по умолчанию.
- **FooterHtml** - общий нижний колонтитул в виде html страницы. В ней должен присутствовать теги **!DOCTYPE**, **html**, **body**
- **HeaderHtml** - общий верхний колонтитул в виде html страницы. В ней должен присутствовать теги **!DOCTYPE**, **html**, **body**
- **FirstFooterHtml** - первый нижний колонтитул в виде html страницы. В ней должен присутствовать теги **!DOCTYPE**, **html**, **body**
- **FirstHeaderHtml** - первый верхний колонтитул в виде html страницы. В ней должен присутствовать теги **!DOCTYPE**, **html**, **body**

- **EvenFooterHtml** - нижний колонтитул в виде html страницы для четных страниц. В ней должен присутствовать теги **!DOCTYPE, html, body**
- **EvenHeaderHtml** - верхний колонтитул в виде html страницы для четных страниц. В ней должен присутствовать теги **!DOCTYPE, html, body**
- **OddFooterHtml** - нижний колонтитул в виде html страницы для нечетных страниц. В ней должен присутствовать теги **!DOCTYPE, html, body**
- **OddHeaderHtml** - верхний колонтитул в виде html страницы для нечетных страниц. В ней должен присутствовать теги **!DOCTYPE, html, body**
- **FooterHeight** - высота нижнего колонтитул в **px**
- **HeaderHeight** - высота верхнего колонтитул в **px**
- **FromEmail** - источник адреса электронной почты отправителя
- **MobileView** - флаг, что у формы есть мобильное представление
- **OnTop** - флаг, что текущая форма будет идти в начале списка форм для печати
- **OnlySigned** - флаг, что форма будет использовать только для подписанных вложенный (только для **штампов**)
- **PageBottomMargin** - нижний отступ от края страницы, задается в **cm**.
- **PageHeight** - высота страницы, задается в **cm**.
- **PageLeftMargin** - левый отступ от края страницы, задается в **cm**.
- **PageOrientation** - ориентация страниц формы. Может иметь значения: **0** - автоматическая (в зависимости от размеров содержимого страницы, если очень большая, то разбивается постранично), **1** - портретная, **2** – альбомная, **3** - масштабируемая (в зависимости от размеров содержимого страницы, если очень большая, то масштабируется).
- **PageRightMargin** - правый отступ от края страницы, задается в **cm**.
- **PageSize** - формат страницы. Если не задан формат страницы, то используется ее размер. Пример форматов: A4, A3 и др.
- **PageTopMargin** - верхний отступ от края страницы, задается в **cm**.
- **PageWidth** - ширина страницы, задается в **cm**.
- **PageZoom** - масштаб страниц. **Не используется**, значение **1**.
- **PricetagHeight** - ширина ценника, задается в **cm**.
- **PricetagMargin** - расстояние между ценниками, задается в **mm**.
- **PricetagWidth** - ширина ценника, задается в **cm**.
- **NamePattern** - шаблон имени формы.
- **PdfA1** - флаг, что форма при генерации будет сконвертирована в формат Pdf/A-1
- **Replicated** - флаг, что форма является тиражируемой, т.к. создается для каждой записи RecordSet-а (перед применением нужно проконсультироваться с [Ильичевым Ильей](#))
- **Version** - версия формы. **Не используется**

Binding - набор параметров, содержащий в себе данные по отображению формы в реестре.

- **BindingID** - уникальный идентификатор привязки формы. Для новой формы задается случайным образом.
- **Client** - идентификатор клиента. **0** - форма доступна всем клиентам, **-1** - форма относится к ФЭД формату
- **Person** - идентификатор пользователя.
- **Source** - идентификатор операции, к которой привязана форма.
- **Endpoint** - идентификатор реестра, в котором находится форма. Совпадает с полем **Name** для соответствующей реестру карты.
- **ViewCondition** - условие доступности формы.
- **ViewConditionName** - имя условия доступности формы. На текущий момент **не используется**, значение "".
- **DefaultOutputFormat** - формат документа. **0** – pdf, **1** – docx, **2** – xlsx.
- **OutputType** - формат печати документа. **0** - обычный, **1** - автоматический, **2** - на этапе, **3** – письмо.
- **Visible** - флаг видимости формы.

- **Rollback** - флаг, содержащий в себе сведения что форма является "откаченной" к типовой. Актуален только для **типовых измененных** форм.
- **Deleted** - флаг, обозначающий что форма помещена в корзину.
- **CheckForPrint** - флаг отображения формы в списке печати.
- **ComponentID** - список UUID идентификаторов компонент. Его нужно узнавать у ответственного за функционал, чтобы избежать разночтений. Идентификатор можно найти в хранилище компонентов в ста <https://git.sbis.ru/app-market/applications>) или в к консоли при открытии компонента в ответе от метода Read \ ReadByComponentUUID (Настройки-Компоненты).

File - набор параметров, содержащий в себе информацию о файле формы. Актуально только для форматов **docx** и **xlsx**

- **Identity** - уникальный идентификатор файла формы. Для новой формы задается случайным образом.
- **Format** - формат документа. **0** – pdf, **1** – docx, **2** – xlsx.
- **Type** - текстовое обозначение формата документа. Может быть pdf, docx, xlsx.